

ÇUKUROVA UNIVERSITY

FACULTY OF ENGINEERING



DEPARTMENT OF COMPUTER ENGINEERING

CEN317 ENGINEERING DESIGN

INTERIM REPORT

WEB-BASED CAR RENTAL PLATFORM WITH FLEET MANAGEMENT AND LLM-BASED RECOMMENDATIONS

2022556033 - METİN IŞIKCAN
2023555001 - GİZEM ACER
[STUDENT NUMBER] - ENES [SURNAME]

NOVEMBER

2026

ADANA

INTERIM REPORT ABSTRACT

In this project we are building a web platform where customers can search for cars and rent them, and rental companies can manage their own cars. Customers can filter the car list by brand, category, transmission, fuel type, number of seats and price. In the next milestones they will also be able to pick a date range and see only the cars that are free in that range, pay with a simulated card form, filter cars on a map and ask a large language model (LLM) for a car suggestion.

The application is written in Python with FastAPI and uses a PostgreSQL database. Pages are created on the server with Jinja2 templates and styled with Bootstrap. The code is split into three layers: routers handle HTTP requests, services hold the business rules and the database queries, and models describe the database tables. To stop the same car from being booked twice for overlapping dates, we put an exclusion constraint on the reservation table in PostgreSQL instead of checking this only in the application code.

We split the work into three milestones. Milestone 1 is finished: users can sign up and log in as a customer or as a company, customers can list and filter cars and open a details page, and companies can add, edit and deactivate their own cars. A seed script loads 30 demo cars, and 67 automated tests run on every pull request. Reservations, payment simulation, the map and the LLM recommendation are planned for the remaining milestones.

Keywords: car rental, web application, FastAPI, PostgreSQL, large language models

CONTENTS

	Page
INTERIM REPORT ABSTRACT	**i**
CONTENTS	**ii**
ABBREVIATIONS AND SYMBOLS	**iii**
LIST OF TABLES	**iv**
LIST OF FIGURES	**v**
1. PROJECT INTRODUCTION	**1**
2. RELEVANT STUDIES AND LITERATURE REVIEW	**2**
3. PROJECT MATERIALS	**3**
3.1. Software and Tools	3
3.2. System Architecture	3
3.3. Data Model	4
3.4. Methods	4
3.4.1. Concurrency Control	4
3.4.2. LLM-Based Recommendation	4
3.5. Development Process	4
4. CURRENT STATE OF THE PROJECT	**5**
5. FUTURE IMPROVEMENTS FOR THE PROJECT	**8**
ACKNOWLEDGEMENTS	**9**
REFERENCES	**10**
ABOUT THE AUTHORS AND AUTHOR CONTRIBUTIONS	**11**
APPENDICES	**12**

ABBREVIATIONS AND SYMBOLS

API	Application Programming Interface
CI	Continuous Integration
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
LLM	Large Language Model
MVP	Minimum Viable Product
ORM	Object-Relational Mapping
PR	Pull Request
SQL	Structured Query Language
TL	Turkish Lira

LIST OF TABLES

Table	Page
Table 3.1. Software, versions and purposes	**3**
Table 4.1. Pages completed in Milestone 1	**5**

LIST OF FIGURES

Figure	Page
Figure 3.1. Use case diagram	**4**
Figure 4.1. Car list with filters	**5**
Figure 4.2. Car details page	**6**
Figure 4.3. “My fleet” page of a company	**6**
Figure 4.4. Add car form	**7**

1. PROJECT INTRODUCTION

Renting a car online is very common today. Big rental companies have their own booking websites, and comparison sites show offers from many companies in one list. Most of these sites let the user choose dates and filter the results, but they are different in what they show and in how much control a small rental company has over its own cars.

In this project we build a simpler version of such a platform as a school MVP. There are two kinds of users. A customer searches for a car, looks at its details and, in the next milestones, rents it for a date range with a fake card payment. A company is a user who adds its own cars, edits them and removes them from the list when needed. On top of this, we wanted two parts that are interesting from an engineering side: making sure two customers cannot book the same car for the same days, and suggesting cars with an LLM based on the customer's budget, number of passengers and purpose of travel.

The project has seven required features: car filtering, date range selection, payment simulation, fleet management, map-based filtering, concurrency control and LLM recommendation. Our goal is to have all seven working by the final presentation. We prefer simple and correct code over adding many extra features.

The rest of the report is organised as follows. Section 2 looks at similar platforms and at some studies on concurrency control and LLM-based recommendation. Section 3 describes the software, the architecture and the data model. Section 4 explains what works right now, and Section 5 lists the work left until the final presentation.

2. RELEVANT STUDIES AND LITERATURE REVIEW

Similar platforms. We looked at three car rental services. Hertz Türkiye rents out the company's own cars with a four-step booking flow: dates and location, vehicle group, extras, and a review before payment. Cars are shown by category, and the user does not get many filters (Hertz Türkiye, n.d.). Turo is a peer-to-peer marketplace where people list their own cars. After choosing a location and dates, users can filter by price, vehicle type, make, year, seats and fuel type (Turo, n.d.). Rentalcars.com collects offers from many suppliers in one list and lets users filter by supplier, transmission, category, price, rating and fuel type (Rentalcars.com, n.d.). Our filters are close to Turo and Rentalcars.com, and our company role is closer to Turo, where each owner manages only their own cars. Sahu et al. (2024) describe a web-based vehicle rental system with a vehicle provider role and machine learning based recommendations, which is quite similar to our project in scope.

Concurrency control. If two customers try to book the same car for overlapping dates at the same moment, a simple "check first, then insert" approach can let both bookings through. Chaudhry and Yousaf (2022) review concurrency control methods and the problems that show up when many transactions work on the same data. PostgreSQL has explicit table and row locks, but its documentation warns that explicit locking can increase the risk of deadlocks (PostgreSQL Global Development Group, n.d.-b). Another option is an exclusion constraint. It makes sure that no two rows meet a given condition at the same time, and it can be limited to some rows with a WHERE clause (PostgreSQL Global Development Group, n.d.-a). Together with range types such as DATERANGE and the overlap operator, it can reject overlapping reservations directly in the database (PostgreSQL Global Development Group, n.d.-c). We chose this approach because the database does the check for us and we do not have to manage locks ourselves.

LLM-based recommendation. LLMs can give answers that sound correct but are not based on real data. Fan et al. (2024) survey retrieval-augmented LLMs, which first fetch relevant data and then let the model write an answer from that data, to reduce such mistakes. Zhu et al. (2025) combine an LLM with collaborative filtering for conversational recommendation and report better results than several baselines. Our planned design uses the same idea in a simple form: the database first selects the cars that fit the customer's budget and passenger count, the LLM only ranks these candidates, and the car ids it returns are checked against the candidate list.

3. PROJECT MATERIALS

This section describes the software we use, how the application is organised, the database tables and the methods we use for the two harder features. We did not use any special hardware or external dataset; the demo data is created by our own seed script.

3.1. Software and Tools

The project is a Python web application with server-rendered HTML and a PostgreSQL database. Table 3.1 lists the software we use. The Python package versions are taken from the project's dependency list.

Table 3.1. Software, versions and purposes.

Software or tool	Version	Purpose
Python	3.13	Programming language
FastAPI	0.143.0	HTTP routes, form handling and dependencies
Uvicorn	0.54.0	Application server
SQLAlchemy	2.1.4	Database access and ORM models
PostgreSQL	16	Database and reservation constraint
psycopg	3.3.6	PostgreSQL driver
Jinja2	3.1.6	Server-rendered HTML templates
pydantic-settings	2.15.0	Settings from environment variables
python-multipart	0.0.32	Reading form data
pwdlib (Argon2)	0.3.1	Password hashing
itsdangerous	2.2.0	Signed session cookies
Bootstrap	5.3	Page layout and form styling
Leaflet	1.x	Interactive map with OpenStreetMap tiles (planned)
pytest	9.1.1	Automated tests
Ruff	0.17.0	Code formatting and linting
uv, Docker Compose, Git, GitHub	–	Dependencies, database setup, version control and code review

3.2. System Architecture

The system is a single web application organised in three layers. Routers handle HTTP requests, check the user's role and return a page or a redirect. Services contain the business rules and the database queries. Models define the database tables. A request always goes from the router to the service and then to the models, never the other way. Keeping the queries out of the routers made the code easier to test and easier to split between team members.

During development PostgreSQL runs in Docker, and the application runs locally with uv. A logged-in user is kept in a signed session cookie. Each protected page states which role (customer or company) may open it; a user who is not logged in is sent to the login page and a user with the wrong role gets an error. Figure 3.1 shows the use case diagram with the two actors, Customer and Company, and the LLM service as a secondary actor.

[Use case diagram will be added here from docs/diagrams/use-case.png]

Figure 3.1. Use case diagram.

3.3. Data Model

The database has three tables: *users*, *cars* and *reservations*. The users table stores account details, a password hash and the role (customer or company). A company is simply a user with a company name, so we did not need a separate company table. Each car belongs to a company through the owner_id column and stores its brand, model, year, category, seats, transmission, fuel type, daily price, city, map coordinates and an active flag. Each reservation links a customer to a car and stores the rental period as a DATERANGE, the total price, the status and only the last four digits of the card. Prices are stored as decimal numbers in TL, never as floating point numbers. Nothing is deleted: cars are deactivated and reservations are cancelled.

3.4. Methods

3.4.1. Concurrency Control

The reservation table has an exclusion constraint named no_double_booking (Appendix A, models.py). It rejects a new confirmed reservation if the same car already has a confirmed reservation whose period overlaps with it. The period is half-open, so a car returned on the 13th can be picked up again on the 13th, and a cancelled reservation does not block the dates anymore. The constraint was created together with the tables in Milestone 1. In Milestone 2 the booking service will simply try to insert the reservation; if PostgreSQL rejects it, the service catches the error and shows the customer that the car has just been taken.

3.4.2. LLM-Based Recommendation

This part is planned for Milestone 3. The customer enters a budget, the number of passengers and the purpose of the trip. An SQL query first selects the active cars that fit the budget and have enough seats. Only this short list is sent to the LLM, which ranks the cars and writes a short reason for each one. The returned car ids are checked against the list before anything is shown, so the model cannot suggest a car that does not exist. If the LLM call fails, the page falls back to the cheapest matching cars.

3.5. Development Process

The work is split into small task cards, and each card is one branch and one pull request. Every pull request is reviewed by the team lead before it is merged. A CI pipeline on GitHub runs code formatting, linting and the full test suite against a real PostgreSQL database for every pull request. We also used AI coding assistants. A shared instruction file in the repository keeps the generated code in the same style, and each pull request states which tool was used and what was changed by hand. Each card also has a few “understanding questions” that the author answers in their own words in the pull request.

4. CURRENT STATE OF THE PROJECT

Milestone 1 is complete. A user can register as a customer or as a company, log in and log out. Pages are protected by role: a customer who opens a company page gets an error, and a user who is not logged in is sent to the login page. Table 4.1 lists the pages built in this milestone.

Table 4.1. Pages completed in Milestone 1.

Page	Who uses it	What it does
Sign up, Log in	Everyone	Creates an account or logs in
Cars	Everyone	Lists active cars with six filters
Car details	Everyone	Shows one car and the company offering it
My fleet	Company	Lists the company's own cars with their status
Add car, Edit car	Company	Adds a new car or changes an existing one

The car list shows only active cars, sorted by price. It can be filtered by brand, category, transmission, fuel type, minimum seats and a price range (Figure 4.1). The selected filters stay in the address bar, so a search can be shared as a link. The filter query is built step by step in the car service (Appendix A, cars.py). Each car has a details page with its features and the company that offers it (Figure 4.2).

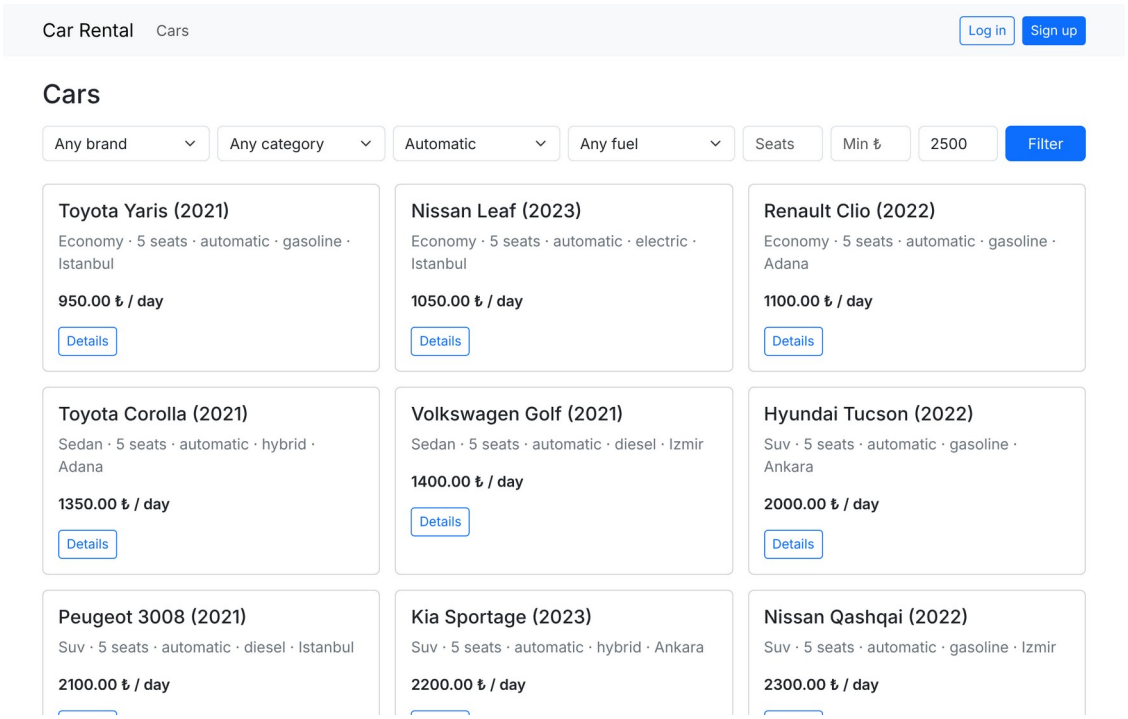


Figure 4.1. Car list with filters (automatic cars up to 2500 TL per day).

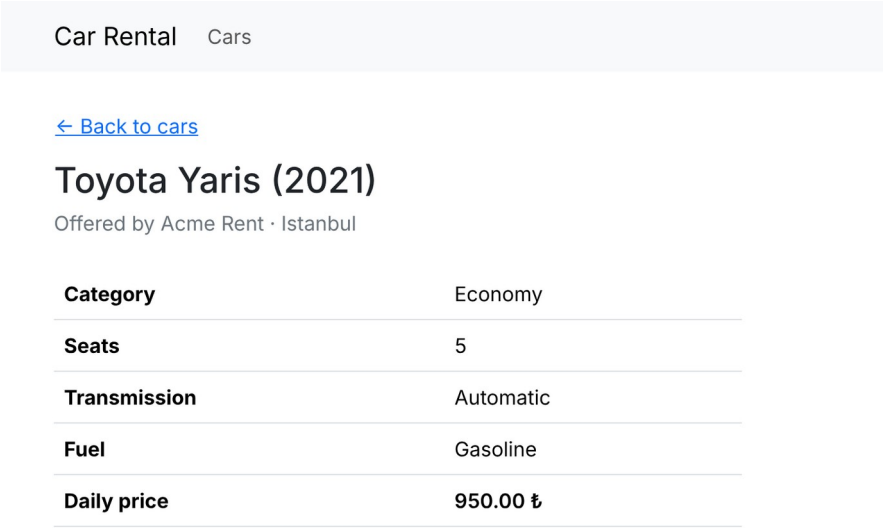


Figure 4.2. Car details page.

A company sees only its own cars on the “My fleet” page (Figure 4.3). It can add a car with a form (Figure 4.4), edit it later and deactivate it. A deactivated car disappears from the customer list but stays in the company’s fleet, so it can be activated again. If a company tries to open another company’s car, the page answers “not found”, so it cannot even tell whether that car exists (Appendix A, fleet.py).

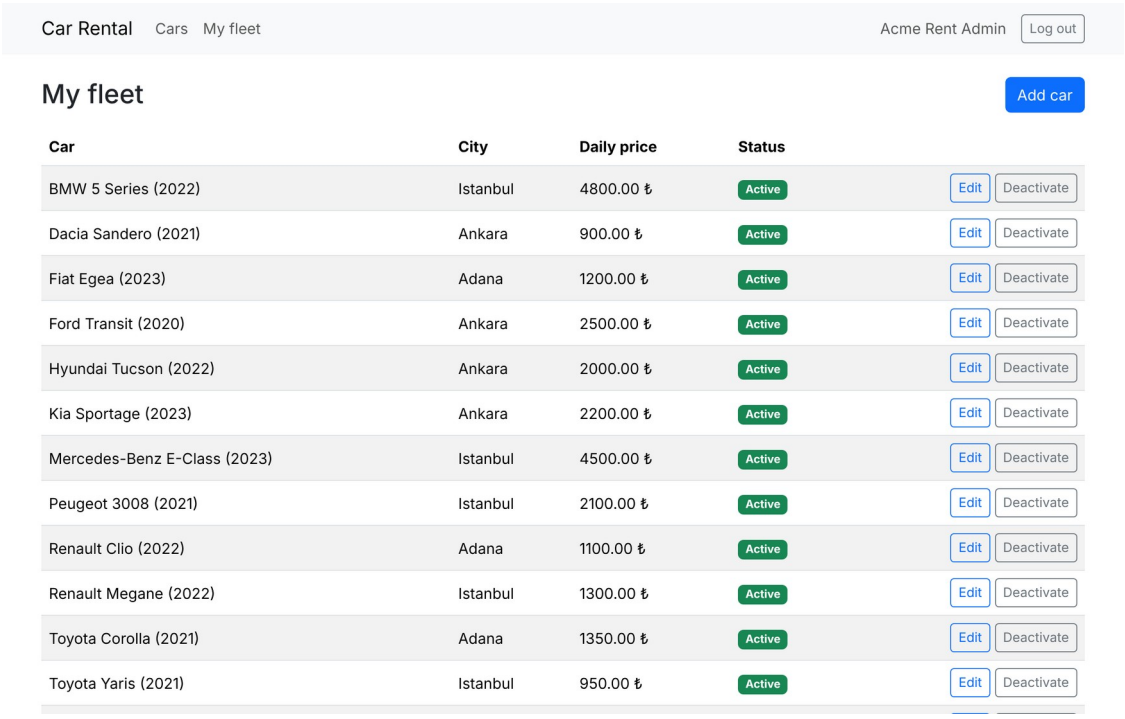


Figure 4.3. “My fleet” page of a company.

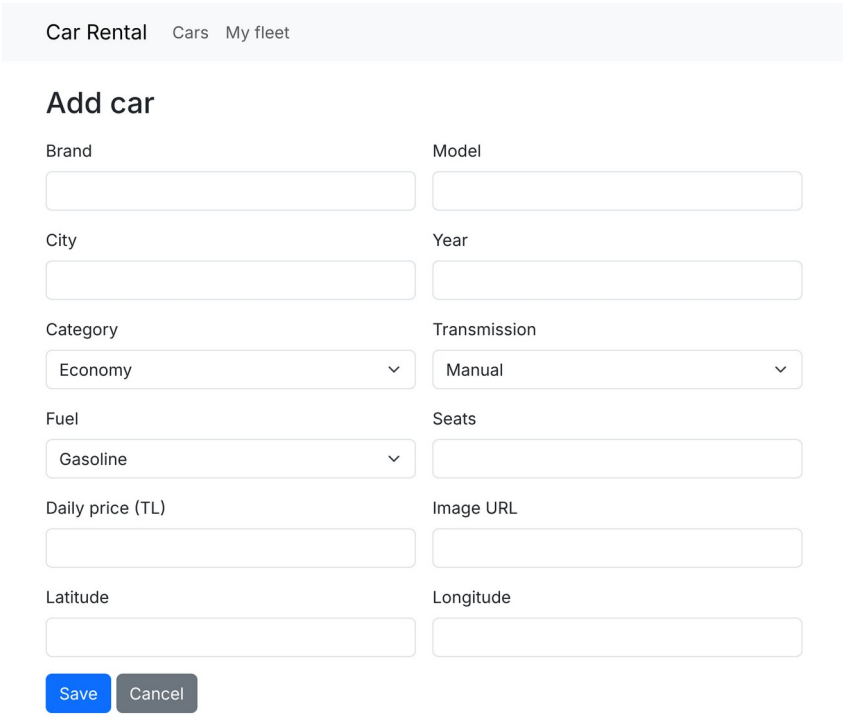


Figure 4.4. Add car form.

All form input is checked on the server, not only in the browser, and invalid input shows a readable error page instead of raw JSON. A seed script resets the database and loads demo data: three companies, one customer and 30 cars in five cities, chosen so that every filter option returns at least one car. The test suite has 67 automated tests that run against a separate test database, so the demo data is never touched by the tests. The source code and the folder structure are given in the appendices.

5. FUTURE IMPROVEMENTS FOR THE PROJECT

The work left until the final presentation is listed below. Milestone 2 covers the first three items and Milestone 3 covers the rest.

- **Date range and availability:** the customer chooses pick-up and return dates and sees only the cars that are free in that range. The car details page will also show the total price for the chosen dates.
- **Reservation and payment simulation:** a fake card form where only the last four digits are stored. Customers can see and cancel their own reservations, and companies can see the reservations for their cars.
- **Concurrency control:** the booking service will rely on the no_double_booking constraint and show a clear message when a car has just been taken by someone else. We also plan a test where two bookings for the same dates are sent at the same time.
- **Map-based filtering:** a Leaflet map with a “Search this area” button that filters the cars by the visible map area.
- **LLM recommendation:** the customer enters a budget, number of passengers and purpose, and the system suggests suitable cars as described in Section 3.4.2.
- **Report:** adding the use case diagram and an architecture figure, and adding the new pages to the final report.

ACKNOWLEDGEMENTS

We would like to thank the instructors of the CEN317 Engineering Design course for their guidance and feedback. This project did not receive any financial or institutional support.

REFERENCES

Chaudhry, N., & Yousaf, M. M. (2022). Concurrency control for real-time and mobile transactions: Historical view, challenges, and evolution of practices. *Concurrency and Computation: Practice and Experience, 34*(3), Article e6549. https://doi.org/10.1002/cpe.6549

Fan, W., Ding, Y., Ning, L., Wang, S., Li, H., Yin, D., Chua, T.-S., & Li, Q. (2024). A survey on RAG meeting LLMs: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (pp. 6491–6501). Association for Computing Machinery. https://doi.org/10.1145/3637528.3671470

Hertz Türkiye. (n.d.). *Hertz Türkiye* [Website]. Retrieved October 21, 2026, from https://www.hertz.com.tr

PostgreSQL Global Development Group. (n.d.-a). *CREATE TABLE* (PostgreSQL 18 documentation). Retrieved October 21, 2026, from https://www.postgresql.org/docs/current/sql-createtable.html

PostgreSQL Global Development Group. (n.d.-b). *Explicit locking* (PostgreSQL 18 documentation, Section 13.3). Retrieved October 21, 2026, from https://www.postgresql.org/docs/current/explicit-locking.html

PostgreSQL Global Development Group. (n.d.-c). *Range types* (PostgreSQL 18 documentation, Section 8.17). Retrieved October 21, 2026, from https://www.postgresql.org/docs/current/rangetypes.html

Rentalcars.com. (n.d.). *Cheap car rental, price match guarantee*. Retrieved October 21, 2026, from https://www.rentalcars.com

Sahu, A., Katlana, A., Sharma, A., Baniya, M. Y., Sureliya, S., & Kanungo, S. (2024). Vehicle rental management system: OnDemand Vehicle Ride. In *2024 International Conference on Advances in Computing Research on Science Engineering and Technology (ACROSET)* (pp. 1–11). IEEE. https://doi.org/10.1109/ACROSET62108.2024.10743592

Turo. (n.d.). *Turo* [Website]. Retrieved October 21, 2026, from https://turo.com

Zhu, Y., Wan, C., Steck, H., Liang, D., Feng, Y., Kallus, N., & Li, J. (2025). Collaborative retrieval for large language model-based conversational recommender systems. In *Proceedings of the ACM on Web Conference 2025* (pp. 3323–3334). Association for Computing Machinery. https://doi.org/10.1145/3696410.3714908

ABOUT THE AUTHORS AND AUTHOR CONTRIBUTIONS

Metin Işıkcan is a Computer Engineering student at Çukurova University and the team lead. He planned the project, split the work into task cards and wrote the shared coding rules for the team. He set up the project skeleton, the Docker database, the GitHub repository and the CI pipeline. He built the database models with the double-booking constraint, the test setup, registration and login with role-based access, the shared page template and error page, the car list with filters and the first version of the seed script. He also built the edit car page, the activate/deactivate button and the car details page, prepared the APA reference list, and reviewed and merged every pull request.

Gizem Acer is a Computer Engineering student at Çukurova University. She prepared the demo data: she extended the seed script to three companies and 30 cars across five cities and made sure that every filter option has results. She also collected and summarised the sources for the literature review on similar platforms, concurrency control and LLM-based recommendation.

Enes [SURNAME] is a Computer Engineering student at Çukurova University. He built the fleet management features for companies: the fleet service with the car form validation, the “My fleet” page, the add car form, and the ownership check, update and activate/deactivate functions, together with their tests. He also wrote the first draft of the Project Materials section of this report.

APPENDICES

Appendix A: Source Code

The full source code is available at https://github.com/Pyro137/car-rental-school-project. The README.md file explains how to set up and run the project and gives a short scenario to try it from start to end. The three parts below are referred to in Sections 3 and 4.

app/models.py (reservation table with the no_double_booking constraint)

```
class Reservation(Base):
    __tablename__ = "reservations"
    __table_args__ = (
        ExcludeConstraint(
            ("car_id", "="),
            ("period", "&&"),
            where=text("status = 'confirmed'"),
            using="gist",
            name="no_double_booking",
        ),
    )

    id: Mapped[int] = mapped_column(primary_key=True)
    car_id: Mapped[int] = mapped_column(ForeignKey("cars.id"))
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), index=True)
    period: Mapped[Range[date]] = mapped_column(DATERANGE)
    status: Mapped[str] = mapped_column(default="confirmed")
    total_price: Mapped[Decimal] = mapped_column(Numeric(10, 2))
    card_last4: Mapped[str] = mapped_column(String(4))
```

app/services/cars.py (car list with filters)

```
def list_cars(db: Session, filters: CarFilters) -> Sequence[Car]:
    stmt = select(Car).where(Car.is_active).order_by(Car.daily_price)
    if filters.brand:
        stmt = stmt.where(Car.brand == filters.brand)
    if filters.category:
        stmt = stmt.where(Car.category == filters.category)
    if filters.transmission:
        stmt = stmt.where(Car.transmission == filters.transmission)
    if filters.fuel:
        stmt = stmt.where(Car.fuel == filters.fuel)
    if filters.min_seats is not None:
        stmt = stmt.where(Car.seats >= filters.min_seats)
    if filters.min_price is not None:
        stmt = stmt.where(Car.daily_price >= filters.min_price)
    if filters.max_price is not None:
        stmt = stmt.where(Car.daily_price <= filters.max_price)
    return db.scalars(stmt).all()
```

app/services/fleet.py (ownership check)

```
def get_owned(db: Session, owner_id: int, car_id: int) -> Car | None:
    car = db.get(Car, car_id)
    return car if car and car.owner_id == owner_id else None
```

Appendix B: Folder Structure

car-rental-school-project
->app
    ->main.py (application setup)
    ->models.py (database tables)
    ->routers (HTTP layer: auth, cars, fleet)
    ->services (business rules and queries)
    ->templates (HTML pages)
    ->static
->scripts
    ->seed.py (demo data)
->tests (automated tests)
->docs (plans, task cards, report drafts)
->docker-compose.yml
->pyproject.toml
->README.md